



FOOD FUSION — CODE EXPLANATION GUIDE

Master File-by-Line Breakdown & 30-Second Examiner Q&A Defense

⚡ GOLDEN RULE FOR ANSWERING SIR / EXAMINER (30 Secs)

When the teacher or examiner points to any file and asks "What does this code do?", answer using this 3-step formula:

1. **1. Name Technology:** "Sir, this file is written in [Python / TypeScript / React]..."
2. **2. State Main Purpose:** "Its main role is to manage [REST API / Cart State / Coupon Validation / Live Tracking]..."
3. **3. Explain Mechanics:** "It uses [Flask handlers / LocalStorage hooks / Array filters / Dynamic hashing] to process data."

🔗 1. BACKEND API — server.py

Presenter: Divesh Salve (Leader)

Role: Python REST API server running on port 5000 that processes HTTP requests from the React frontend web app.

```
# CORS Headers (Prevents browser security blocks)
def send_cors_headers(handler):
    handler.send_header('Access-Control-Allow-Origin', '*')
    handler.send_header('Access-Control-Allow-Methods', 'GET, POST, OPTIONS')

# GET Handler (do_GET)
if parsed_path.path == '/api/health':
    # Returns {"status": "online", "app": "Food Fusion API"}
elif parsed_path.path == '/api/dishes':
    # Returns list of food dishes formatted as JSON

# POST Handler (do_POST)
if parsed_path.path == '/api/orders':
    # Parses order payload & generates Order ID: ORD_XXXXXX
```

- **CORS Fix:** Allows `http://localhost:8080` frontend to talk to `http://localhost:5000` backend.
- **JSON Serialization:** Uses `json.dumps()` to format Python data structures into clean JSON strings.

2. DATA SCHEMAS & ARRAYS — `src/lib/data.ts`

Presenter: Divesh / Rashmin

Role: Stores data models and arrays for food items, categories, and hotel partners.

- **categories Array:** Stores category keys (biryani, indian, pizza, burger, chinese, healthy, dessert).
- **dishes Array:** Contains object schemas storing id, name, desc, price, veg, category, calories, time, image.
- **Helper Functions:** `getRestaurant(id)` uses `.find()`; `getRestaurantDishes(id)` uses `.filter()`.

3. SHOPPING CART STATE — `src/lib/cart.ts`

Presenter: Rashmin Oak

Role: Manages global cart state and client-side LocalStorage persistence.

- **useCart() Hook:** Reactive hook maintaining array of { dish: Dish, quantity: number }.
- **LocalStorage Sync:** Saves cart data under key `online_food_cart_v2` so cart persists on page refresh.
- **Currency Formatter:** `inr(amount)` formats numbers into Indian Rupees (₹379).

4. MENU & FILTERING — `src/routes/menu.tsx`

Presenter: Rashmin Oak

Role: Renders interactive menu, category tabs, veg/non-veg filter, and image lightbox modal.

```
const list = useMemo(() => {
  return menuDishes.filter((dish) => {
    if (mode === "veg" && !dish.veg) return false;
    if (mode === "nonveg" && dish.veg) return false;
    if (cat !== "all" && dish.category !== cat) return false;
    return !q || dish.name.toLowerCase().includes(q.toLowerCase());
  });
}, [mode, cat, q, menuDishes]);
```

Key Concept: Uses `useMemo` for instant array filtering without unnecessary page re-renders.

5. CHECKOUT & COUPON ENGINE — `src/routes/checkout.tsx`

Presenter: Divesh Salve

Role: Validates address, applies coupons, simulates Razorpay payment, and auto-triggers 3D Spin Wheel popup.

- **handleApplyCoupon():** Checks codes (FLASH60, QUIZ100, FREEDEL) with \$0\$ minimum subtotal; falls back to dynamic 20% discount.
- **Auto Spin Popup useEffect:** `placed = true` automatically opens Spin & Win rewards modal after 500ms.

6. ORDER TRACKING & RIDER HASHING — src/routes/orders.tsx

Presenter: Divesh Salve

Role: Displays order history, live status timeline, and dynamically assigned delivery rider.

```
function getRiderForOrder(orderId: string) {
  let hash = 0;
  for (let i = 0; i < orderId.length; i++) {
    hash = orderId.charCodeAt(i) + ((hash << 5) - hash);
  }
  const index = Math.abs(hash) % RIDERS.length;
  return RIDERS[index];
}
```

Rider Assignment: Hashes unique orderId string into numeric index to consistently assign riders (e.g. *Rashmin Salve, Naman Sharma*).

? TOP 10 EXAMINER QUESTIONS & 30-SECOND ANSWERS

Q1: Where is your database stored?

Answer: "Sir, we use Supabase Cloud Database (PostgreSQL) for real-time order sync and hotel partner accounts. For instant client-side persistence like cart state and sessions, we use browser LocalStorage."

Q2: How does Python backend communicate with React frontend?

Answer: "Sir, server.py runs an HTTP REST API server on port 5000. React sends fetch() requests to endpoints like /api/dishes and /api/orders. CORS headers are configured in Python to prevent security blocks."

Q3: How does category filtering work on the Menu page?

Answer: "Sir, we use React's useMemo hook in menu.tsx. When a user selects a category like Biryani (cat=biryani), it executes an array .filter() on data.ts instantly without re-rendering the whole page."

Q4: How do you assign delivery riders to orders?

Answer: "Sir, in orders.tsx we built a string hashing algorithm getRiderForOrder(orderId). It converts the Order ID into a numeric index using bitwise shifting to assign a rider consistently from our pool."

Q5: Is your payment gateway real or simulated?

Answer: "Sir, it is an interactive Razorpay Test Mode simulation in checkout.tsx. It supports test UPI IDs, card validation, and Cash on Delivery with full receipt generation."

Q6: How does the 3D Spin & Win Fortune Wheel work?

Answer: "Sir, in rewards-modal.tsx we calculate random degree rotations (rotation + 1440 + prizeIndex * 60) with CSS cubic-bezier transitions to simulate realistic 3D wheel physics."

Q7: How does the AI Support Chatbot handle voice commands?

Answer: "Sir, it uses the browser's native Web Speech API — webkitSpeechRecognition for speech-to-text input and speechSynthesis for text-to-speech voice output."

Q8: What is the difference between Customer Login and Hotel Login?

Answer: "Sir, Customer Login allows users to view order history and saved addresses, while Hotel Login (/hotel/login) grants access to the Hotel Partner Dashboard to manage dish availability and order progress."

Q9: Why did you choose React + Vite instead of plain HTML?

Answer: "Sir, React provides component-based architecture and state management, while Vite ensures instant hot module reloading, superfast build performance, and TypeScript type safety."

Q10: How do you handle dark mode styling?

Answer: "Sir, we use custom Tailwind CSS design tokens with glassmorphism effects, HSL tailwind colors, and backdrop-blur overlays to create a sleek dark theme."